Follow the Money Without Moving the Data

Privacy-limited questions between banks, planned by an LLM agent, for anti-money-laundering

Team *Cache me if you can*: Abhishek, Bhavya, Shreyas Deb, Akshat Srivastava
LMA Group Project 2026

Code: https://github.com/BhavyaV05/Transaction-Thieves (see `README.md` for reproduction)

11 November 2026

Abstract

A bank sees only the transactions that touch its own customers, and launderers exploit this by routing money through several banks. We ask a narrow question: if a bank cannot settle an alert on its own, can it gain by asking other banks a few small, typed questions, each answered with a single differentially private number, while no transaction record ever leaves the other bank? We built a full system around that idea: a question language that partner banks execute exactly on their own ledgers, a privacy accountant, a compliance gate, statistical back-testing that a question must pass before it may influence a decision, a fitted evidence-fusion formula, and an LLM planner (Gemma 4 12B, orchestrated with LangGraph) that chooses or writes the questions. We reproduced or re-implemented the main detection models (GFP + LightGBM, Multi-GNN, MEGA-GNN, FraudGT), federated baselines (DP-FedAvg, layer-wise exchange) and thirteen published LLM agents, and evaluated everything under one pre-registered, frozen protocol on IBM's synthetic AMLworld data. On the held-out test split (771 askable alerts, privacy budget $\varepsilon = 2$ per alert) our system closes -0.007 $[-0.038, 0.026]$ of the gap between one bank and a model that sees every bank: the questions do not measurably help. It does clearly beat the strongest external LLM agent, BED-LLM (H9, Holm $p = 0.004$), and several of its own design choices (back-testing, free composition of questions) measurably help inside the system. We report this negative headline result as it stands and explain why we think it happens.

1 Introduction

Money laundering is a network crime. A typical scheme moves money through many accounts, often at different banks, so that no single institution sees the whole pattern. Altman et al. showed on synthetic data that a detection model trained on one bank's slice of the transaction graph performs far worse than the same model trained on the whole graph [1]. Real banks cannot simply pool their data. In the US, section 314(b) of the PATRIOT Act lets banks share information about suspected laundering [30], and Singapore's COSMIC platform does something similar [31], but in practice these requests are slow, manual and usually involve sending raw records.

We study a middle ground. When a bank's own model cannot decide an alert, an agent at that bank (the *home bank*) may ask a partner bank a typed question such as "did the receiving account pass on at least 80% of what it received within 24 hours?". The partner runs the question exactly on its own ledger and replies with one noisy value (yes/no, a count, or a private set-intersection count) under differential privacy [13]. No rows leave the partner bank. The home bank combines the answers with its own score and decides.

Three ideas make this more than a toy:

1. **Back-testing before trust.** Every question, including those an LLM invents on the spot, must first show on past alerts that its answer is informative (a likelihood ratio, with false-discovery control using e-values [14]). An answer that has not passed this test cannot move the decision.

2. **A formula decides, not the LLM.** The LLM plans what to ask, but the final probability is computed by a fitted fusion formula from numbers only. This is a direct response to Sharma's observation that an LLM investigator made worse calls than the model it was meant to help because it trusted its own fluent reasoning [6].
3. **One gate for everyone.** Our planner and every external agent we compare against pass through the same compliance gate, privacy accountant and simulated bank network, with the same LLM backbone and call limits. Only the control strategy differs.

Our research questions were: (RQ1) how much of the one-bank-to-see-everything gap can private questions close; (RQ2) does an LLM planner beat simple planners at choosing questions; (RQ3) does back-testing matter; and (RQ6) how does the system compare with published AML and general-purpose LLM agents. All hypotheses (H1–H10), thresholds and the Holm correction order were pre-registered and frozen with a hash before the test split was opened.

Contributions. (i) An end-to-end, privacy-accounted research harness for cross-bank questions, with 30 simulated partner banks; (ii) faithful reproductions of the main AML detection baselines on a time-respecting split; (iii) ports of thirteen LLM agents into a single fair harness; (iv) a pre-registered evaluation, run once on the test split, whose mostly negative result we report in full.

2 Literature Review

AML data and detection models. AMLworld [1] is a synthetic multi-bank transaction dataset from IBM with ground-truth laundering labels and pattern types. Its reference model is gradient-boosted trees on features from the Graph Feature Preprocessor (GFP) [2]. Later work reports higher scores with graph neural networks: Multi-GNN [3] (edge classification on directed multigraphs), MEGA-GNN [4] (multi-edge aggregation) and the graph transformer FraudGT [5]. All of these either see one bank or the whole graph; none asks what a bank could learn by querying others while their data stay in place. We use them as the two ends of our scale.

Federated and private cross-bank learning. Project Aurora [9] and Tian et al. [10] study collaboration across institutions; Ceydeli et al. [11] exchange GNN layer activations between clients to detect cross-client patterns. Federated averaging with differentially private training is the standard private baseline. These approaches are all-or-nothing: banks train jointly, rather than asking per-alert questions. For set questions we follow the differentially private set-intersection cardinality protocol of Purcell et al. [12], simulating its noisy output.

LLM agents for fraud and AML. Sharma [6] combines graph features, explanations and an agentic case investigator; Torres et al. [7] triage AML alerts with subgraph evidence and counterfactual checks; FinFRE-RAG [8] retrieves similar fraud cases; Co-Investigator is a SAR-drafting pipeline. These agents work inside one institution and do not test their evidence before using it.

General agents and information seeking. ReAct [19], Reflexion [20], ReWOO [21], LATS [22] and LLMCompiler [23] are general tool-use loops; DSPy [24] lets us tune a ReAct agent's prompts with MIPROv2 so that it is not a straw man. BED-LLM [17] and Uncertainty of Thoughts [18] choose what to ask next by expected information gain, the closest idea to ours, but they were not tested where every question costs privacy budget and answers are noisy and delayed.

Keeping an LLM's ideas only if they pass a test. CAAFE [16] keeps LLM-proposed features that improve validation scores; POPPER [15] falsifies hypotheses with e-values. We apply the same principle to LLM-written questions, using e-BH false-discovery control [14]. Our strategy library follows ReasoningBank [25], and the evidence retriever combines a SubgraphRAG-style edge scorer [26] with G-Retriever's prize-collecting Steiner tree [27].

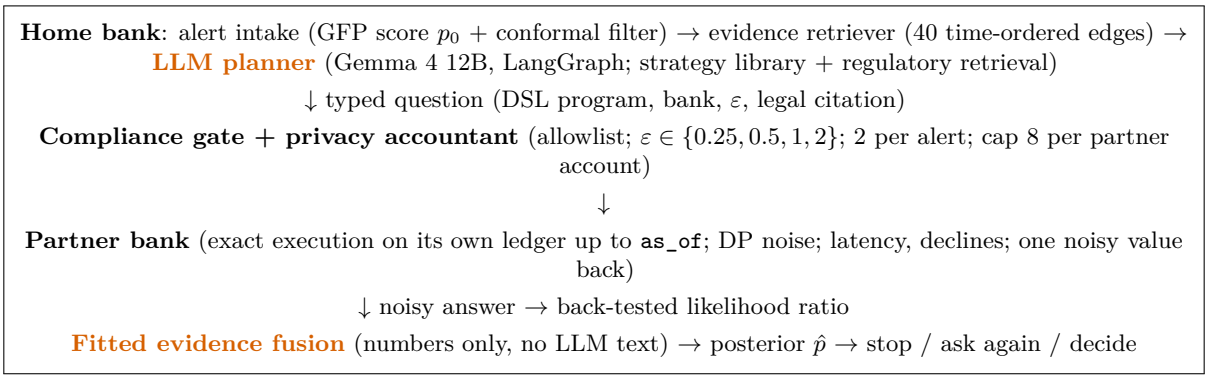


Figure 1: One alert's path through the system. No transaction record leaves a partner bank, and the LLM never writes the final probability.

3 Dataset Created / Used

Source data. We use IBM AMLworld [1], high-illicit-ratio variants **HI-Small** (about 5 million transactions, 0.5 million accounts) and **HI-Medium** (about 32 million transactions, 2 million accounts). Laundering is rare: about one transaction in a thousand. HI-Medium is the headline dataset; HI-Small was used for development and for baselines that do not scale.

What we built on top. The raw data are one global ledger, so we created the multi-bank setting ourselves:

- **Bank views.** The 30 largest banks each get a private view containing only transactions that touch their customers. Each one is both a possible home bank and a simulated partner with its own ledger, reply latency (usually under a day, up to four days, timeout at five), declines and an optional free-text note (an untrusted channel, used in our prompt-injection tests).
- **Time-respecting split.** Days 1–9 train, 10–12 validation, 13–16 test. Every tool takes the alert time `as_of` and cannot see later data; the test split is locked in code (`FTM_FINAL_RUN`) and was opened only by the frozen final-run script.
- **Alerts.** The home bank's GFP model scores every transaction; a class-conditional conformal filter passes only transactions the model cannot confidently settle. On HI-Medium test this gives **4,478 alerts**: 3,873 in the main period (975 laundering, 2,552 clusters of linked alerts) and 605 in the sparse tail days, reported separately.
- **Askable alerts.** A question can only be answered if a partner bank holds the relevant counterparty. Of the 3,873 headline alerts, **771** are askable (241 positives). A pre-registered random subset of 1,000 test alerts contains 208 askable ones, used for the more expensive arms.
- **Back-test table and strategy library.** From training-period alerts we built a table of question programs and privacy levels with their back-tested likelihood ratios (440 rows, 23 accepted under e-BH on HI-Medium), and a library of question strategies indexed by the *shape* of the local graph, never by account.

4 Methodology

Figure 1 shows the pipeline. We describe each part briefly.

Questions and privacy. A question is a program in a small domain-specific language whose slots are relative to the alert (e.g. `passed_on(ALERT.receiver, window=after(24), min_frac=0.8)` or `overlap(ALERT.receiver, dir=out, among=ALERT.payers)`), never a raw account number or date. The partner answers with randomised response (yes/no), a Laplace-noised count, or a private

set-intersection count. Privacy is protected per transaction. The accountant enforces a budget of $\varepsilon = 2$ per alert and a cap of $\varepsilon = 8$ on each partner account across the whole test period (basic composition).

Gate. Every question, from our planner or from an external agent, passes the same allowlist: known partner, allowed ε level, program parses, no aliases, numbers or dates, a valid legal basis cited from a small regulatory corpus (314(b), MAS COSMIC), and enough budget. LLMs only see random aliases of accounts.

Back-testing. For each (program, ε) pair we replay the question on past training alerts, with the same noise, and estimate how much more likely each answer is for laundering than for clean alerts. A pair is *accepted* only if it passes e-value FDR control [14]; the likelihood ratio is also capped at the ceiling implied by ε. New LLM-written programs go through the same test before their answer may count.

Fusion. The posterior is $\operatorname{logit} \hat{p} = \operatorname{logit} p_0 + \sum_k w_{f(k)} \log \mathrm{LR}_k$, with one weight per question family, fitted with ridge and shrinkage on validation and frozen. During validation we found that the naive version ($w = 1$) moves p_0 the wrong way, because the questions partly re-measure what p_0 already knows (double counting); the fitted conditional fusion was the fix, frozen before the test run.

Planner. The planner is a LangGraph state machine around Gemma 4 12B (4-bit, served with vLLM). At each step it sees aliased evidence, p_0, the current posterior, the remaining budget and a menu of library strategies. It either *selects* a listed question, *composes* a new one in the DSL, or stops. Its free text (hypotheses, reasoning) is logged but never enters the decision. We picked the backbone with a pre-registered size sweep (E18, Section 5).

Non-LLM planners and baselines. To test whether an LLM is needed at all, the same harness runs a random planner, a fixed expert order, an expected-information-gain (EIG) planner, a learned policy, and an enumerator that tries every expressible question. *Raw sharing* lets the home bank see the actual records of three partner banks with no noise, a non-private reference.

External agents. We ported thirteen published agents into the same harness: Sharma's investigator (official MIT code), Torres et al., Co-Investigator, FinFRE-RAG, BED-LLM, Uncertainty of Thoughts, multi-agent debate, ReAct/Reflexion/ReWOO/LATS/LLMCompiler, and a DSPy-tuned ReAct. Each runs *natively* (as published, one bank's view) and *ported* (given our question tool and budget). We also built *adapted* arms that run an agent's control loop inside our system, to separate "better control" from "knowing the likelihood ratios".

Evaluation protocol. The main metric is the Brier score of $\hat{p}$ and the *gap closed*

$$G = \frac{B(\text{one bank}) - B(\text{system})}{B(\text{one bank}) - B(\text{see everything})},$$

where "see everything" is GFP trained on all banks' data. $G = 0$ means no gain over the bank alone and $G = 1$ means as good as pooling all data. Intervals are 95% cluster bootstraps (clusters of linked alerts), and p-values are Holm-adjusted within the primary family (H1–H4, H9) and the secondary family (H6, H7, H10). All thresholds, the equivalence margin (0.01 Brier) and the choice of best external agent were fixed on validation and frozen with a SHA-256 hash.

5 Experiment Runs

The project was compute-heavy. The final test run alone used **44 GPU-hours** on an RTX 5070 Ti (5,440 LLM verdict rows, plus 6,600 CPU verdict rows), and the validation reruns on the rewritten LangGraph pipeline took roughly 60 more GPU-hours. Detection models trained for days on the 5070 Ti and on a 3070 Ti laptop (FraudGT ran 500 epochs), and some arms ran on the *ada* HPC cluster. The main runs were:

Reproducing the detection baselines. We reproduced GFP + LightGBM on the paper's own split (F1 62.95 vs. 62.86 ± 0.25 published; XGBoost 62.51 vs. 63.23) and Multi-GNN with the official code (F1 67.64 vs. 68.16 ± 2.65). MEGA-GNN was only partially reproduced (59.7 vs. 74.88 after 10 of 80 epochs, compute-limited), and FraudGT ran its full 500 epochs on our split (best val F1 0.686). We then retrained all of them, one-bank and see-everything, on our time-respecting split. One important finding: the official GNN loaders sample future edges, so their see-everything scores are not valid upper limits under a time guard. Re-scored time-respecting, Multi-GNN's HI-Medium PR-AUC drops from 0.611 to 0.199. GFP is therefore our upper limit.

Federated baselines. DP-FedAvg (Flower + Opacus) at client level and edge level, with ε from 1 to 16 plus a no-noise arm, three seeds each, and the layer-wise exchange of Ceydeli et al. in eight arms. On HI-Small validation, the best layer-wise arm (Sync-SGD, forward + backward exchange) slightly beat one-bank GFP (Brier 0.001116 vs. 0.001167); DP-FedAvg at a meaningful ε did not reach one-bank GFP, and edge-level DP-FedAvg collapsed to the base rate.

Backbone sweep (E18). Seven open models were compared as planners on 160 validation alerts, then the best three on 460. Gemma 4 12B won (Brier 0.1741 vs. 0.1844 for Qwen3.8-27B and 0.1890 for Qwen3-14B). gpt-oss-20b was dropped for invalid outputs, and a "thinking" arm was stopped because it spent its whole token budget reasoning without answering.

Fine-tuning (E16, E10). We distilled the teacher's successful planning steps into a QLoRA [28, 29] student (Qwen3-8B, LoRA rank 16, three seeds, 33 minutes each) and trained a Qwen3-4B "System 1" fast model for routine typed decisions with conformal cascade bounds. The students learned the format (token accuracy 0.93–0.94) but planned worse than the 12B teacher (Brier 0.223–0.226 vs. 0.201 on 160 val alerts). The fast model kept every cascade error bound and, used as an injection screen, cut ReAct's prompt-injection success from 5–8% to 0%.

Validation of all agents. All 33 systems ran on validation fit, calibration and evaluation parts, DSPy prompt optimisation ran for 2.7 GPU-hours, and the RAG and injection benchmarks ran on the HPC cluster.

The final run. After the freeze (hash `b094561b...`), `run_final.py` ran each arm once on the test split: our system at $\varepsilon \in \{1, 2, 4\}$, four ablations, five non-LLM planners, raw sharing, BED-LLM on all askable alerts, and Co-Investigator plus two adapted agents on the subset. Because of the deadline we ran a pre-registered reduced plan; the full plan was estimated at about 188 GPU-hours.

6 Evaluations

Detection models. On all 3,873 headline alerts, one-bank GFP has Brier 0.1585 (PR-AUC 0.602); see-everything GFP has 0.0854 (PR-AUC 0.848). This is the gap we try to close, and it is large: at a review capacity of 10% of alerts, the see-everything model's precision is 0.98 against 0.72.

Table 5 in the appendix puts every system we ran on test side by side on every metric; Tables 6 and 7 do the same for all detection models and all thirteen LLM agents on validation.

	Hypothesis (test)	Estimate [95% CI]	p_{Holm}	Verdict
H1	Gap closed vs. one bank	−0.007 [−0.038, 0.026]	1.00	not supported
H2	Benefit relative to raw sharing	−0.165 [−2.88, 0.75]	1.00	not supported
H3	LLM planner beats simple planners	−0.001 [−0.006, 0.003]	0.33	not supported
H4	Back-testing helps (+ FDR audit)	−0.005 [−0.010, −0.000]	0.072	not supported
H9	**Beats best ported external (BED-LLM)**	**0.266** [−0.022, 0.799]	**0.004**	**supported**
H7	Formula vs. LLM deciding	−0.000 [−0.004, 0.004]	1.00	not supported

Table 1: Pre-registered hypotheses on the test split (771 askable alerts; H9's interval is for the gap-closed difference). H5, H6, H8 and H10 were not run in the reduced plan.

Group	System	Brier	Gap closed [95% CI]	Qs/alert	ε spent
Reference	One bank only	0.1668	0	–	–
	See everything (GFP)	0.0966	1	–	–
	Raw records, 3 banks (no privacy)	0.1640	0.040 [−0.107, 0.161]	1.99	0
Non-LLM planners	Random	0.1698	−0.042 [−0.102, 0.005]	5.12	1.83
	Fixed expert order	0.1694	−0.036 [−0.072, −0.012]	2.33	1.89
	Expected info. gain	0.1823	−0.220 [−0.392, −0.114]	8.80	1.44
	Learned policy	0.1814	−0.207 [−0.369, −0.103]	7.99	1.45
	Enumerator	0.1708	−0.057 [−0.178, 0.055]	1.78	0.69
External	BED-LLM (frozen pick)	0.1956	−0.410 [−0.831, −0.176]	2.76	0.61
Ours	**Compose,** $\varepsilon = 2$	**0.1673**	**−0.007** [−0.038, 0.026]	1.55	1.46
	Budget $\varepsilon = 1$	0.1666	0.004 [−0.025, 0.031]	–	0.74
	Budget $\varepsilon = 4$	0.1679	−0.015 [−0.057, 0.030]	–	2.67

Table 2: Test results on the 771 askable alerts (lower Brier is better).

Main result (H1, H2). Table 2 shows the headline. Our system's Brier (0.1673) is statistically the same as the bank working alone (0.1668): it closes essentially none of the gap. Even raw, noise-free records from three partner banks close only 4% of the gap, and its interval also covers zero. This tells us the limit is not mainly privacy noise: on these alerts, the evidence a partner bank can supply about one counterparty adds little beyond what GFP already computes from the home bank's graph. The privacy curve is flat ($\varepsilon = 1$, 2 and 4 are within 0.0013 Brier of each other), which is consistent with that reading.

Planners (H3). The LLM planner is better than every simple planner except the enumerator; the EIG planner and the learned policy, which ask many questions, are clearly worse. The test required beating the enumerator too, so H3 is not supported.

External agents (H9). This is where the system clearly wins (Table 3, Figure 2). On all 771 askable alerts, ours beats BED-LLM, the external agent frozen on validation as the strongest, by 0.27 in gap closed (Holm $p = 0.004$). BED-LLM and Co-Investigator make the final call themselves and drift away from the well-calibrated p_0; Co-Investigator gives no usable verdict on 71% of alerts. The adapted agents, which run inside our system and use our fusion, come much closer, which suggests that most of our advantage comes from back-tested likelihood ratios plus a formula rather than from a cleverer control loop. Scored on questions only (the same fusion applied to every agent's questions), all systems are far below zero, so none of them, ours included, asks questions that are worth much on their own.

Cost. Our planner uses 2.4 LLM calls, about 12.9k tokens and 29 GPU-seconds per alert, and spends $\varepsilon \approx 1.46$ of its budget of 2. BED-LLM uses three times the calls for a worse result.

Agent (subset, 208 askable)	Brier	Gap closed [95% CI]	LLM calls	GPU-s/alert	No-verdict
Ours, compose	**0.1576**	**0.005** [−0.044, 0.058]	2.4	28.4	0
DSPy ReAct (adapted)	0.1623	−0.051 [−0.149, 0.033]	1.4	7.4	0
ReWOO (adapted)	0.1638	−0.068 [−0.145, −0.007]	0.9	4.1	0
BED-LLM	0.1805	−0.262 [−0.797, 0.026]	6.9	32.9	0
Co-Investigator	0.2334	−0.878 [−1.709, −0.471]	7.6	38.7	0.71

Table 3: External agents against our system on the same 208 askable subset alerts, same backbone, gate and budget.

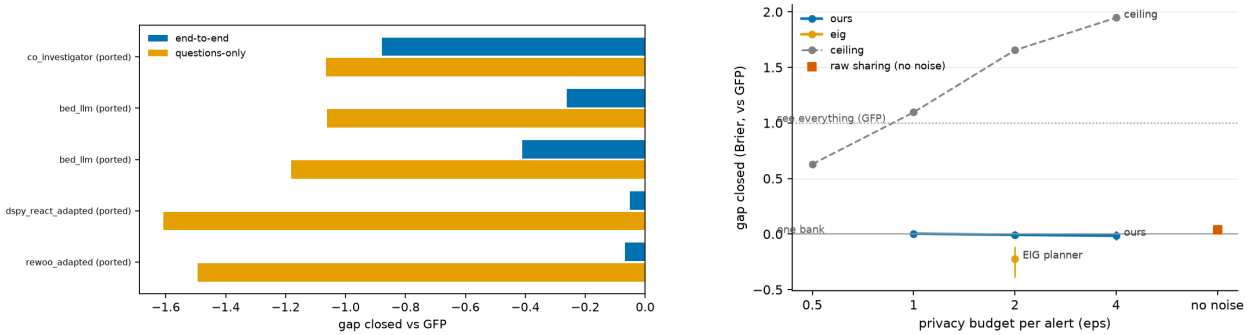


Figure 2: Left: gap closed by each external agent and by our system (end-to-end and questions-only). Right: privacy curve of our system on the 771 test alerts; the curve is flat.

Robustness checks. On *all* 3,873 frozen alerts (non-askable alerts keep p_0), ours differs from one bank by +0.0001 [−0.0003, 0.0005] Brier: we do no harm. BED-LLM is worse by +0.0057 [0.003, 0.008] and Co-Investigator by +0.0157. Broken down by laundering pattern, our system helps slightly on cycles and fan-in and hurts slightly on bipartite patterns, all within noise.

7 Ablations

We switched off one component at a time on the same 208 askable subset alerts (Table 4).

Ablation	Brier	Δ vs. full system [95% CI]	Qs/alert	Reading
Full system (compose)	0.1576	–	1.53	
LLM may only pick from a fixed list	0.1656	+0.0080 [0.0031, 0.0131]	1.18	**writing questions helps**
No back-testing (LLM's own belief)	0.1626	+0.0049 [0.0003, 0.0099]	2.71	**back-testing helps**
Act only, no stated hypothesis	0.1592	+0.0016 [−0.0026, 0.0059]	1.46	small, not significant
LLM decides when to stop/escalate	0.1577	+0.0001 [−0.0035, 0.0038]	1.52	no difference

Table 4: Ablations (positive Δ means the ablation is worse than the full system).

Two components clearly earn their place. Letting the LLM *compose* new questions, rather than choose from a fixed menu, lowers Brier by 0.008, with an interval that excludes zero. Removing *back-testing* makes the system ask almost twice as many questions, costs twice the LLM calls and GPU time, and is worse by 0.005. In the pre-registered H4 test on all 771 alerts the back-testing effect was −0.005 [−0.010, −0.000] (Holm $p = 0.072$), and H4 also required that a false-discovery audit of accepted questions pass, which it did not (0.714). Stating a hypothesis before acting and letting the LLM handle the stop/escalate decision made no measurable difference (H7). The privacy budget ablation ($\varepsilon = 1, 2, 4$) is in Table 2. From validation we also have: naive fusion (all weights 1) moves the score the wrong way, which is why we fitted the fusion weights; and the QLoRA student and Qwen3-8B prompted are both worse than the Gemma 4 12B teacher.

8 Limitations

- **Synthetic data, simulated banks.** AMLworld is generated, and the partner banks, latency and private set intersection are simulated. Real 314(b) traffic and real laundering are messier.
- **Few askable alerts.** Only about 20% of alerts have a counterparty that any partner bank can answer about, which caps the possible gain and makes intervals wide (771 alerts, 241 positives).
- **Privacy unit.** Our guarantee is per transaction, not per customer; the per-account cap of $\varepsilon = 8$ is a partial patch.
- **Reduced final run.** Because of time we ran a pre-registered subset of arms: H5, H6, H8 and H10, the evidence-view arms, the fine-tuned students and $\varepsilon = 0.5$ and ∞ were not run on test. Only one external agent ran on all askable alerts.
- **Recorded deviations.** The see-everything GFP rerun on HI-Medium did not exactly reproduce its earlier validation scores (largest difference 0.935); we accepted it with a recorded deviation and archived the scores read-only so it can be rerun. HI-Medium layer-wise exchange was dropped and HI-Small kept with a deviation (val Spearman 0.93–0.96 against a 0.95 gate). The CPU planner arms were rerun once with the frozen recalibrated p_0. FraudGT was scored once before the freeze. The pre-registration had no TA countersignature, and a few test-path bugs (PRs #150, #152, #154, #156, #157) were fixed during the final run without changing any frozen setting.
- **Only a time-respecting upper limit.** The GNN see-everything models use future edges in their official loaders, so only GFP is used as an upper limit.

9 Conclusions: what we learned from a system that did not work

The honest summary is that the idea did not work. Under a frozen, pre-registered protocol, an LLM agent asking partner banks privacy-limited questions did not measurably improve on the bank's own GFP model (H1), and only one of our primary hypotheses held (H9, against other LLM agents). We think the reasons are more useful than the headline, so we list what we learned.

1. The partner banks had little to add that the home bank did not already know. Even *raw, noise-free* records from three partner banks closed only 4% of the gap, with an interval covering zero. The gain of the see-everything model comes from multi-hop graph features computed over the whole network, while one partner can only describe one counterparty one hop away. GFP's own features (fan-in, fan-out, counts, cycles in the local graph) already summarise most of that hop. Our questions largely *re-measured* what p_0 knew. This is the single biggest lesson: before building the asking machinery, we should have measured how much information a partner's view adds beyond the home bank's features (a raw-sharing upper bound) and stopped or redesigned if it was small.

2. Back-tested likelihood ratios did not transfer to the alerts we cared about. On validation, the realised slope of the labels on the summed log-likelihood ratios was about zero ($\hat{\lambda} = -0.03$ [-0.94, 0.75] for compose; 1 would mean "as claimed"), and most question families pointed the right way only about half the time. The ratios were estimated on historical training-period alerts, but the alerts that reach the agent are a selected, harder population (the conformal filter keeps exactly the cases the model finds ambiguous), and the data drift over time. A ratio that is correct on average can be uninformative on the residual cases. A naive fusion with weight 1 therefore moved scores the *wrong* way; fitting the weights only shrank the damage to zero.

3. Too few alerts could be asked about. Only about 20% of alerts (771 of 3,873) had a counterparty at a partner bank. Even a perfect system could move at most a fifth of the headline,

and with 241 positives the intervals are wide: our minimum detectable effect was larger than the effects we saw.

4. Differential privacy caps how much one answer can say. With $\varepsilon \leq 2$ per alert, a yes/no answer can shift the odds by at most e^{ε}; the largest accepted log-ratio was 1.34, and most were far smaller. The flat privacy curve ($\varepsilon = 1, 2, 4$) shows privacy was not the binding constraint here, but in a setting where the answers were informative it would be.

5. LLMs are poor probability judges, but they can be useful planners if they are kept away from the decision. Every published agent that writes its own probability ranked alerts worse than p_0 on validation (AUC 0.29–0.72 against 0.76; ReAct answered 0.85 for 62% of alerts), and on test BED-LLM and Co-Investigator were clearly worse than the bank alone. The same control loops run inside our system with fusion (the *adapted* arms) matched p_0. Our safeguards (back-testing plus a formula) are why our system never hurt the base model, and why H9 holds. Inside the system, letting the LLM compose new questions and filtering them by back-testing both helped measurably, so the LLM's planning was not worthless, it just had little signal to plan with.

6. Small data does not make a good student, and bigger is not always better. The QLoRA student learned the output format from 252 examples but not the policy, and the 12B backbone beat a 27B one. With so little real signal in the answers, there was little for fine-tuning to learn.

7. Engineering lessons. Pre-registration and a test lock kept us honest when the results were disappointing, and we would do it again. But the time between "first real end-to-end number" and the freeze was too short: the key diagnostic (point 2) only appeared late in validation. The official GNN loaders leak future edges, which silently inflates see-everything baselines; anyone using them as an upper limit should re-score with a time guard. Reproducing every baseline first was worth the GPU days: it is how we found that leak.

What we would do next. Measure partner information first; design questions about what the home bank *cannot* compute (multi-hop, cross-bank cycles via private set intersection over several banks); estimate likelihood ratios on the same selected alert population the agent sees; and increase coverage with more partner banks.

Reproducibility

Everything is in the repository:
https://github.com/BhavyaV05/Transaction-Thieves.
The `README.md` explains setup, data preparation, the validation runs, how the frozen final run is launched (`scripts/run_final.py`) and how the analysis is regenerated (`scripts/final_analysis.py`); `REVIEW.md` maps the repository. The pre-registration is in `prereg/` with its freeze hash, and the test tables are archived in `docs/results/test_analysis/`.

References

[1] E. Altman et al. Realistic synthetic financial transactions for anti-money laundering models. NeurIPS Datasets and Benchmarks, 2023.

[2] J. Blanuša et al. Graph Feature Preprocessor: real-time subgraph-based feature extraction for financial crime detection. arXiv:2402.08593, 2024.

[3] B. Egressy et al. Provably powerful graph neural networks for directed multigraphs (Multi-GNN). AAAI, 2024.

[4] H. Ç. Bilgi et al. Multigraph message passing with bi-directional multi-edge aggregations (MEGA-GNN). arXiv:2412.00241, 2024.

[5] J. Lin et al. FraudGT: a simple, effective, and efficient graph transformer for financial fraud detection. ICAIF, 2024.

[6] R. Sharma. Toward auditable fraud detection: combining graph features, model explanations, and agentic case investigation. arXiv:2607.19266, 2026.

[7] D. Torres, W. Cheng, K. Hu. Explainable AML triage with LLMs: evidence retrieval and counterfactual checks. arXiv:2604.19755, 2026.

[8] X. Tan, Y. Ma, X. Zhang. Understanding structured financial data with LLMs: a case study on fraud detection (FinFRE-RAG). ACL, 2026.

[9] Bank for International Settlements. Project Aurora: the power of data, technology and collaboration to combat money laundering across institutions and borders. 2023.

[10] Z. Tian et al. Towards collaborative anti-money laundering among financial institutions. WWW, 2025.

[11] Ceydeli, Wang, Atasu. Federated layer-wise exchange for cross-client money laundering pattern detection. arXiv:2605.06433, 2026.

[12] C. Purcell et al. Split, count, and share: a differentially private set intersection cardinality estimation protocol. UAI, 2023.

[13] C. Dwork, A. Roth. The algorithmic foundations of differential privacy. Foundations and Trends in TCS, 2014.

[14] R. Wang, A. Ramdas. False discovery rate control with e-values. JRSS-B, 2022.

[15] K. Huang et al. Automated hypothesis validation with agentic sequential falsifications (POPPER). ICML, 2025.

[16] N. Hollmann, S. Müller, F. Hutter. Large language models for automated data science: CAAFE. NeurIPS, 2023.

[17] D. Choudhury et al. BED-LLM: intelligent information gathering with LLMs and Bayesian experimental design. arXiv:2508.21184, 2025.

[18] Z. Hu et al. Uncertainty of Thoughts: uncertainty-aware planning enhances information seeking in LLMs. NeurIPS, 2024.

[19] S. Yao et al. ReAct: synergizing reasoning and acting in language models. ICLR, 2023.

[20] N. Shinn et al. Reflexion: language agents with verbal reinforcement learning. NeurIPS, 2023.

[21] B. Xu et al. ReWOO: decoupling reasoning from observations for efficient augmented language models. arXiv:2305.18323, 2023.

[22] A. Zhou et al. Language Agent Tree Search (LATS). ICML, 2024.

[23] S. Kim et al. An LLM compiler for parallel function calling. ICML, 2024.

[24] O. Khattab et al. DSPy: compiling declarative language model calls into self-improving pipelines. ICLR, 2024.

[25] S. Ouyang et al. ReasoningBank: scaling agent self-evolving with reasoning memory. ICLR, 2026.

[26] M. Li, S. Miao, P. Li. Simple is effective: the roles of graphs and LLMs in KG-based RAG (SubgraphRAG). ICLR, 2025.

[27] X. He et al. G-Retriever: retrieval-augmented generation for textual graph understanding. NeurIPS, 2024.

[28] E. J. Hu et al. LoRA: low-rank adaptation of large language models. ICLR, 2022.

[29] T. Dettmers et al. QLoRA: efficient finetuning of quantized LLMs. NeurIPS, 2023.

[30] 31 CFR 1010.540. Voluntary information sharing among financial institutions (314(b)). U.S. Code of Federal Regulations.

[31] Monetary Authority of Singapore. Guidelines on COSMIC (Collaborative Sharing of ML/TF Information and Cases). 2024.

A Full comparison tables

Table 5: Every system on the test split against ours, on every metric. HI-Medium, frozen final run g5, budget $\varepsilon = 2$ per alert unless stated. ΔB = Brier minus one bank on the same alerts (negative is better). Gap closed is against GFP see-everything; "QO" scores only the system's questions through our fusion. AUC = ROC-AUC. ECE uses 10 equal-mass bins. P@10%, R@10% = precision and recall when the top 10% of alerts by score are reviewed. No-v. = share of alerts with no usable verdict (scored as p_0). Costs are per alert; tokens are input plus output. Non-LLM planners and raw sharing use no LLM. Confidence intervals for the main rows are in Tables 2–4.

		Probability quality				Ranking and calibration			Review at 10%			Cost per alert					
Group	System	Brier	ΔB	Gap	Gap QO	AUC	PR-AUC	ECE	P@10%	R@10%	No-v.	Qs	ε	LLM calls	Tokens	GPU-s
All 771 askable test alerts (241 laundering)																
Reference	One bank only (GFP p_0)	0.1668	–	0	–	0.809	0.660	0.098	0.83	0.27	–	–	–	–	–	–
	See everything (GFP, upper limit)	0.0966	−0.0702	1.000	–	0.915	0.876	0.055	1.00	0.32	–	–	–	–	–	–
	Raw records, 3 banks, no privacy	0.1640	−0.0028	0.040	–	0.846	0.731	0.140	0.83	0.27	0.00	1.99	0.00	–	–	–
Non-LLM planners	Random	0.1698	0.0029	−0.042	−1.27	0.810	0.665	0.103	0.81	0.26	0.00	5.12	1.83	–	–	–
	Fixed expert order	0.1694	0.0025	−0.037	−1.24	0.807	0.655	0.100	0.83	0.27	0.00	2.33	1.89	–	–	–
	Expected information gain	0.1823	0.0154	−0.220	−1.59	0.795	0.649	0.134	0.81	0.26	0.00	8.80	1.44	–	–	–
	Learned policy	0.1814	0.0145	−0.208	−1.57	0.797	0.651	0.133	0.81	0.26	0.00	7.99	1.45	–	–	–
	Enumerator (every question)	0.1708	0.0040	−0.057	−1.19	0.814	0.670	0.124	0.82	0.26	0.00	1.78	0.69	–	–	–
External LLM agents	BED-LLM	0.1956	0.0288	−0.410	−1.18	0.735	0.553	0.103	0.60	0.19	0.00	2.76	0.61	6.9	21.6k	32.6
Ours and ablations	**Ours: compose,** $\varepsilon = 2$	0.1673	0.0005	−0.007	−1.38	0.816	0.663	0.102	0.83	0.27	0.00	1.55	1.46	2.4	12.9k	29.0
	Ours, $\varepsilon = 1$	0.1666	−0.0003	0.003	−1.32	0.819	0.665	0.104	0.83	0.27	0.00	0.82	0.74	1.2	6.2k	14.1
	Ours, $\varepsilon = 4$	0.1679	0.0011	−0.015	−1.40	0.812	0.664	0.105	0.82	0.26	0.00	2.37	2.67	4.1	22.2k	49.8
Pre-registered subset: 208 askable test alerts (60 laundering)																
Reference	One bank only (GFP p_0)	0.1580	–	0	–	0.825	0.688	0.123	0.86	0.30	–	–	–	–	–	–
	See everything (GFP, upper limit)	0.0722	−0.0858	1.000	–	0.938	0.910	0.042	1.00	0.35	–	–	–	–	–	–
	Raw records, 3 banks, no privacy	0.1537	−0.0044	0.051	–	0.872	0.774	0.162	0.86	0.30	0.00	1.89	0.00	–	–	–
Non-LLM planners	Random	0.1591	0.0011	−0.013	–	0.851	0.724	0.132	0.86	0.30	0.00	5.31	1.85	–	–	–
	Fixed expert order	0.1590	0.0010	−0.011	–	0.829	0.690	0.127	0.86	0.30	0.00	2.57	1.88	–	–	–
	Expected information gain	0.1773	0.0193	−0.225	–	0.815	0.677	0.157	0.86	0.30	0.00	8.88	1.35	–	–	–
	Learned policy	0.1718	0.0138	−0.160	–	0.832	0.696	0.156	0.86	0.30	0.00	8.77	1.36	–	–	–
	Enumerator (every question)	0.1569	−0.0011	0.013	–	0.844	0.713	0.145	0.86	0.30	0.00	1.66	0.67	–	–	–
External LLM agents	BED-LLM	0.1805	0.0225	−0.262	−1.06	0.774	0.608	0.136	0.71	0.25	0.00	2.79	0.67	6.9	21.5k	32.9
	Co-Investigator	0.2334	0.0754	−0.879	−1.06	0.756	0.555	0.219	0.71	0.25	0.71	2.27	0.97	7.6	16.9k	38.7
	DSPy ReAct (adapted)	0.1623	0.0043	−0.051	−1.61	0.832	0.700	0.145	0.86	0.30	0.00	1.30	1.63	1.4	3.0k	7.4
	ReWOO (adapted)	0.1638	0.0058	−0.068	−1.49	0.834	0.692	0.141	0.86	0.30	0.00	0.89	0.85	0.9	1.8k	4.1
Ours and ablations	**Ours: compose,** $\varepsilon = 2$	0.1576	−0.0004	0.005	–	0.838	0.697	0.132	0.86	0.30	0.00	1.53	1.47	2.4	12.9k	28.4
	– fixed list only	0.1656	0.0076	−0.088	−1.54	0.833	0.687	0.141	0.86	0.30	0.00	1.18	1.52	1.2	4.1k	9.0
	– act only (no hypothesis)	0.1592	0.0012	−0.014	−1.24	0.841	0.700	0.136	0.86	0.30	0.00	1.46	1.48	2.0	10.6k	22.4
	– no back-testing	0.1626	0.0046	−0.053	−1.17	0.820	0.682	0.137	0.86	0.30	0.00	2.71	1.32	5.4	27.2k	60.1
	– LLM decides stop/escalate	0.1577	−0.0003	0.004	−1.26	0.833	0.691	0.137	0.86	0.30	0.00	1.52	1.46	2.4	13.1k	29.2

Table 6: Detection models on validation (val `eval` part, all (transaction, bank view) rows; base rate about 0.1%, so Brier values are small). Skill = Brier skill against the constant base rate. FraudGT ran on its own pipeline and rows (best val F1 over 500 epochs), so it is not strictly comparable. †Official GNN loaders sample future edges; time-respecting re-scoring lowers them a lot.

		HI-Small (93,238 rows)				HI-Medium (one bank only)			
View	Model	Brier	Skill	PR-AUC	F1	Brier	Skill	PR-AUC	F1
–	Constant base rate	0.001307	0.000	0.001	–	0.001644	0.000	0.002	–
One bank	GFP + LightGBM (p_0)	0.001167	0.107	0.202	0.283	0.001451	0.117	0.233	0.350
	Multi-GNN	0.001114	0.147	0.241	0.336	0.001330	0.191	0.284	0.369
	MEGA-GNN	0.001082	0.172	0.295	0.345	0.001388	0.156	0.259	0.350
Federated	DP-FedAvg client-level, $\varepsilon = 8$	0.001275	0.025	0.064	0.129	0.001607	0.023	0.064	0.149
	DP-FedAvg client-level, $\varepsilon = 16$	0.001271	0.028	0.078	0.177	0.001597	0.029	0.077	0.167
	DP-FedAvg client-level, no noise	0.001259	0.036	0.083	0.175	0.001569	0.045	0.094	0.195
	DP-FedAvg edge-level, $\varepsilon = 8$	0.001307	0.000	0.001	0.002	0.001644	0.000	0.002	0.004
	DP-FedAvg edge-level, no noise	0.001296	0.009	0.021	0.065	0.001646	−0.002	0.021	0.074
	Layer-wise, local only	0.001290	0.013	0.111	0.143				
	Layer-wise, FedAvg, no exchange	0.001229	0.059	0.142	0.187				
	Layer-wise, FedAvg, bwd exchange	0.001248	0.045	0.147	0.200				
	Layer-wise, FedProx, no exchange	0.001279	0.021	0.117	0.147				
	Layer-wise, FedProx, bwd exchange	0.001280	0.020	0.104	0.163				
	Layer-wise, Sync-SGD, no exchange	0.001200	0.082	0.136	0.186				
	Layer-wise, Sync-SGD, fwd exchange	0.001202	0.080	0.184	0.248				
	Layer-wise, Sync-SGD, fwd+bwd	**0.001116**	**0.146**	0.269	0.328	0.001579	0.039	0.163	0.193
See everything	GFP + LightGBM (upper limit)	0.000906	0.307	0.419	0.474				
	Multi-GNN†	0.000719	0.450	0.548	0.638				
	MEGA-GNN†	0.000821	0.372	0.484	0.560				
	FraudGT (own run)	0.000673	–	0.589	0.686				

Table 7: All LLM agents on validation (HI-Medium val `eval`, 160 askable alerts, same Gemma 4 12B backbone, gate, budget and caps; alerts with a verdict). *Native* = as published, one bank's view; *ported* = given our question tool; *adapted* = the agent's control loop inside our system, with our fusion making the decision. AUC with 95% cluster-bootstrap CI; Brier on the verdict alerts. Our compose system on the same alerts: Brier 0.1907 against p_0 0.1896.

Agent	Verdicts	AUC agent	AUC p_0	AUC difference	Brier agent	Brier p_0
BED-LLM (ported)	160	0.69 [0.56, 0.80]	0.76 [0.66, 0.84]	−0.07 [−0.14, 0.01]	0.2165	0.1896
BED-LLM (adapted)	160	0.75 [0.64, 0.83]	0.76 [0.66, 0.84]	−0.01 [−0.04, 0.01]	0.1971	0.1896
Co-Investigator (native)	51	0.29 [0.15, 0.43]	0.80 [0.62, 0.92]	−0.51 [−0.72, −0.25]	0.4820	0.1637
Co-Investigator (ported)	45	0.54 [0.37, 0.71]	0.70 [0.52, 0.87]	−0.16 [−0.42, 0.11]	0.3929	0.2088
Multi-agent debate (ported)	160	0.41 [0.31, 0.51]	0.76 [0.66, 0.84]	−0.35 [−0.49, −0.21]	0.4168	0.1896
Multi-agent debate (adapted)	160	0.77 [0.67, 0.85]	0.76 [0.66, 0.84]	+0.01 [−0.01, 0.03]	0.1921	0.1896
DSPy ReAct (MIPROv2) (ported)	160	0.47 [0.39, 0.55]	0.76 [0.66, 0.84]	−0.29 [−0.42, −0.15]	0.4456	0.1896
DSPy ReAct (MIPROv2) (adapted)	160	0.77 [0.66, 0.85]	0.76 [0.66, 0.84]	+0.01 [−0.01, 0.03]	0.1891	0.1896
FinFRE-RAG (native)	160	0.72 [0.63, 0.79]	0.76 [0.66, 0.84]	−0.04 [−0.13, 0.06]	0.2796	0.1896
FinFRE-RAG (ported)	157	0.70 [0.62, 0.78]	0.76 [0.65, 0.84]	−0.05 [−0.14, 0.05]	0.3020	0.1909
LATS (ported)	106	0.49 [0.39, 0.57]	0.77 [0.64, 0.87]	−0.28 [−0.43, −0.12]	0.4694	0.1750
LATS (adapted)	160	0.75 [0.64, 0.84]	0.76 [0.66, 0.84]	−0.01 [−0.03, 0.01]	0.1995	0.1896
LLMCompiler (ported)	154	0.45 [0.36, 0.53]	0.75 [0.64, 0.84]	−0.30 [−0.41, −0.17]	0.4729	0.1956
LLMCompiler (adapted)	160	0.77 [0.67, 0.85]	0.76 [0.66, 0.84]	+0.01 [−0.01, 0.02]	0.1902	0.1896
MediQ (adapted)	160	0.76 [0.66, 0.84]	0.76 [0.66, 0.84]	−0.00 [−0.02, 0.02]	0.1927	0.1896
ReAct (ported)	151	0.45 [0.36, 0.54]	0.76 [0.64, 0.84]	−0.31 [−0.45, −0.14]	0.4592	0.1887
ReAct (adapted)	160	0.76 [0.66, 0.84]	0.76 [0.66, 0.84]	+0.00 [−0.01, 0.02]	0.1936	0.1896
Reflexion (ported)	153	0.44 [0.35, 0.53]	0.76 [0.66, 0.85]	−0.32 [−0.46, −0.17]	0.4752	0.1877
Reflexion (adapted)	160	0.75 [0.65, 0.84]	0.76 [0.66, 0.84]	−0.01 [−0.02, 0.01]	0.1976	0.1896
ReWOO (ported)	155	0.47 [0.39, 0.56]	0.75 [0.64, 0.84]	−0.28 [−0.40, −0.12]	0.5214	0.1949
ReWOO (adapted)	160	0.77 [0.68, 0.85]	0.76 [0.66, 0.84]	+0.01 [0.00, 0.03]	0.1876	0.1896
Sharma investigator (native)	160	0.67 [0.58, 0.75]	0.76 [0.66, 0.84]	−0.09 [−0.14, −0.03]	0.2350	0.1896
Sharma investigator (ported)	160	0.65 [0.55, 0.73]	0.76 [0.66, 0.84]	−0.11 [−0.17, −0.05]	0.2408	0.1896
Torres et al. (native)	146	0.58 [0.51, 0.64]	0.76 [0.64, 0.85]	−0.18 [−0.26, −0.08]	0.4413	0.1921
Torres et al. (ported)	148	0.54 [0.46, 0.61]	0.76 [0.65, 0.85]	−0.23 [−0.30, −0.12]	0.4621	0.1903
Uncertainty of Thoughts (ported)	160	0.66 [0.57, 0.75]	0.76 [0.66, 0.84]	−0.10 [−0.21, 0.02]	0.2476	0.1896
Uncertainty of Thoughts (adapted)	160	0.75 [0.65, 0.84]	0.76 [0.66, 0.84]	−0.01 [−0.03, 0.01]	0.1976	0.1896